

Python 小程序 5

以此代码演示了面向对象的继承多态、抽象类、简单工厂模式和集合元组的使用

```
from abc import ABC, abstractmethod
from typing import Union, List

class Person(ABC):
    """人员基类（抽象类）"""
    def __init__(self, name: str, age: int, gender: str):
        self._name = name
        self._age = age
        self._gender = gender
    @abstractmethod
    def get_info(self) -> str:
        """获取人员信息的抽象方法"""
        pass
    def get_name(self) -> str:
        return self._name
    def get_age(self) -> int:
        return self._age
    def get_gender(self) -> str:
        return self._gender

class Student(Person):
    """学生类，继承自 Person"""
    def __init__(self, name: str, age: int, gender: str, student_id: str, major: str):
        super().__init__(name, age, gender)
        self.student_id = student_id
        self.major = major
        self.courses = set()
    def add_course(self, course: str) -> None:
        self.courses.add(course)
    def get_info(self) -> str:
        return f"学生 | 姓名: {self._name}, 年龄: {self._age}, 性别: {self._gender}, 学号: {self.student_id}, 专业: {self.major}, 课程: {self.courses}"

class Teacher(Person):
    """教师类，继承自 Person"""
    def __init__(self, name: str, age: int, gender: str, teacher_id: str, department: str):
        super().__init__(name, age, gender)
        self.teacher_id = teacher_id
        self.department = department
```

```

        self.subjects = tuple()
    def set_subjects(self, subjects: tuple) -> None:
        self.subjects = subjects
    def get_info(self) -> str:
        return f"教师 | 姓名: {self._name}, 年龄: {self._age}, 性别: {self._gender}, 工号: {self.teacher_id}, 部门: {self.department}, 教授科目: {self.subjects}"

class PersonFactory:
    """人员工厂类（简单工厂模式）"""
    @staticmethod
    def create_person(person_type: str, *args) -> Union[Student, Teacher, None]:
        """创建人员对象"""
        if person_type.lower() == 'student':
            return Student(*args)
        elif person_type.lower() == 'teacher':
            return Teacher(*args)
        else:
            return None

class InformationManager:
    """个人信息管理系统类"""
    def __init__(self):
        self.people: List[Person] = []
    def add_person(self) -> None:
        """添加人员"""
        print("\n 请选择要添加的人员类型:")
        print("1. 学生")
        print("2. 教师")
        try:
            choice = int(input("请输入选择: "))
            name = input("请输入姓名: ")
            age = int(input("请输入年龄: "))
            gender = input("请输入性别: ")
            if choice == 1:
                student_id = input("请输入学号: ")
                major = input("请输入专业: ")
                person = PersonFactory.create_person('student', name, age, gender, student_id, major)
                # 添加课程
                courses_input = input("请输入课程（用逗号分隔）: ")
                if courses_input:
                    for course in courses_input.split(','):

```

```

        person.add_course(course.strip())
    elif choice == 2:
        teacher_id = input("请输入工号: ")
        department = input("请输入部门: ")
        person = PersonFactory.create_person('teacher', name, age, gender,
teacher_id, department)
        # 添加教授科目
        subjects_input = input("请输入教授科目（用逗号分隔）: ")
        if subjects_input:
            subjects = tuple(subject.strip() for subject in
subjects_input.split(','))
            person.set_subjects(subjects)
        else:
            print("无效的选择！")
            return
        self.people.append(person)
        print("人员添加成功！")
    except ValueError:
        print("输入格式错误！")
def show_all_people(self) -> None:
    """显示所有人员信息"""
    if not self.people:
        print("暂无人员信息！")
        return
    print("\n" + "="*80)
    print("所有人员信息:")
    print("-"*80)
    for i, person in enumerate(self.people, 1):
        print(f"{i}. {person.get_info()}")
    print("="*80)
def search_people(self) -> None:
    """搜索人员"""
    keyword = input("请输入搜索关键词: ").strip().lower()
    if not keyword:
        print("关键词不能为空！")
        return
    found = []
    for person in self.people:
        if (keyword in person.get_name().lower() or
            keyword in str(person.get_age()) or

```

```

        keyword in person.get_gender().lower() or
        keyword in person.get_info().lower()):
            found.append(person)
    if not found:
        print("未找到匹配的人员！")
        return
    print(f"\n 找到 {len(found)} 条匹配记录:")
    print("-"*80)
    for i, person in enumerate(found, 1):
        print(f"{i}. {person.get_info()}")
    print("-"*80)
def run(self) -> None:
    """运行信息管理系统"""
    while True:
        print("\n===== 个人信息管理系统 =====")
        print("1. 添加人员")
        print("2. 查看所有人员")
        print("3. 搜索人员")
        print("0. 退出系统")
        try:
            choice = int(input("请输入您的选择: "))
            if choice == 0:
                print("感谢使用, 再见!")
                break
            elif choice == 1:
                self.add_person()
            elif choice == 2:
                self.show_all_people()
            elif choice == 3:
                self.search_people()
            else:
                print("无效的选择, 请重新输入!")
        except ValueError:
            print("请输入数字!")
if __name__ == "__main__":
    manager = InformationManager()
    manager.run()

```